

# US-013 — Validate Transaction Date

**Project:** Personal Finance & Money Flow Tracker

**Feature:** Transaction Management

**User Story:** US-013

**Estimated Effort:** 4 hours

**Status:** Implementation Documentation

---

## 1. User Story

**As a user,** I want the application to validate the transaction date when creating a transaction, so that every transaction contains a valid date in the required format and historical transactions can be recorded accurately.

### Acceptance Criteria

1. Transaction date must be provided.
2. Date must follow DD-MM-YYYY.
3. Historical dates must be supported.
4. Invalid calendar dates must be rejected.
5. Date validation must not introduce timezone-related date changes.
6. Invalid transactions must not be saved.
7. Existing API response conventions must be maintained.

---

## 2. Why Date Validation Is Required

A transaction represents a financial activity that occurred on a particular date. The application will eventually use transaction dates for:

- Monthly expense reports.
- Income and expense summaries.
- Historical financial analysis.
- Cash-flow calculations.
- Transaction filtering.
- Balance calculations.

If invalid dates are accepted, reports and date-based filtering may produce incorrect results.

For example, 31-02-2026 has the correct visual structure but is not a real calendar date because February 2026 has only 28 days.

**Therefore, date validation must happen before the transaction is saved.**

---

## 3. Existing Backend Architecture

The project uses the following Apps Script structure:

```
Apps Script Project
■
■■■■ Code.gs
■■■■ Api.gs
■■■■ TransactionService.gs
■■■■ AccountService.gs
■■■■ BalanceService.gs
■■■■ ValidationService.gs
■■■■ SheetRepository.gs
■■■■ ResponseUtil.gs
■■■■ Config.gs
```

File   Responsibility

--- ---

```
`Code.gs`   Application entry point
`Api.gs`   API request routing
`TransactionService.gs`   Transaction business logic
`AccountService.gs`   Account-related operations
`BalanceService.gs`   Balance calculations
`ValidationService.gs`   Input validation
`SheetRepository.gs`   Google Sheets data access
`ResponseUtil.gs`   Standard API responses
`Config.gs`   Application configuration
```

The date validation belongs in `ValidationService.gs` because validation should be centralized and reusable.

---

## 4. Existing TransactionService Structure

The current service object is:

```
const TransactionService = {

  createTransaction: function(data) {
    // To be implemented
  },

  updateTransaction: function(transactionId, data) {
    // To be implemented
  },

  deleteTransaction: function(transactionId) {
    // To be implemented
  },

  getTransactions: function() {
    // To be implemented
  },

  getTransaction: function(transactionId) {
    // To be implemented
  }

};
```

US-013 changes only the `createTransaction()` method.

The implementation should remain inside the existing service object. A separate global `createTransaction()` function should not be maintained because it can create duplicate business logic and uncertainty about which function the API calls.

---

## 5. Date Validation Rules

Rule Requirement

--- ---

Required Yes

Format `DD-MM-YYYY`

Historical dates Allowed

Future dates Allowed by this user story

Invalid calendar dates Rejected

Date conversion Avoid unnecessary conversion

Timezone conversion Must not change the entered date

Storage approach Preserve the validated date string

### Accepted examples

```
14-09-2026
01-01-2025
29-02-2024
31-12-1990
15-08-1995
```

### Rejected examples

```
14/09/2026
2026-09-14
14-9-2026
31-02-2026
29-02-2025
00-12-2026
14-13-2026
```

---

## 6. Why Validate the Date as a String?

The required format is DD-MM-YYYY, for example:

```
14-09-2026
```

JavaScript does not reliably parse arbitrary DD-MM-YYYY strings using:

```
new Date('14-09-2026');
```

A transaction date is also a calendar date, not necessarily a timestamp. Therefore, the implementation should:

1. Validate the input as a string.
2. Check the exact format.
3. Extract day, month, and year.
4. Validate the calendar date.
5. Return the original string without conversion.

This avoids unnecessary timezone-related behavior.

---

## 7. Timezone and Date Conversion Risk

Avoid code such as:

```
var date = new Date(transactionDate);
```

Also avoid converting a date-only value through:

```
Utilities.formatDate(  
    new Date(transactionDate),  
    Session.getScriptTimeZone(),  
    'dd-MM-yyyy'  
);
```

A timestamp may contain timezone information. When converted between UTC and a local timezone, the displayed date can move to the previous or next day.

For US-013, the safer approach is:

```
Input:      14-09-2026  
Validated: 14-09-2026  
Stored:    14-09-2026
```

The validator does not create a JavaScript Date object.

---

## 8. Implementation in ValidationService.gs

### 8.1 Validate required input

```
if (  
    transactionDate === null ||  
    transactionDate === undefined ||  
    transactionDate === ''  
) {  
    return {  
        valid: false,  
        code: 'TRANSACTION_DATE_REQUIRED',  
        message: 'Transaction date is required.'  
    };  
}
```

This rejects null, undefined, and an empty string.

### 8.2 Validate data type

```
if (typeof transactionDate !== 'string') {  
    return {  
        valid: false,  
        code: 'INVALID_TRANSACTION_DATE',  
        message: 'Transaction date must be a string in DD-MM-YYYY format.'  
    };  
}
```

The expected request is:

```
{  
    "transactionDate": "14-09-2026"  
}
```

A numeric value such as 14092026 is rejected.

### 8.3 Validate exact format

```

var datePattern = /^\\d{2}-\\d{2}-\\d{4}$/;

if (!datePattern.test(transactionDate)) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE_FORMAT',
    message: 'Transaction date must be in DD-MM-YYYY format.'
  };
}

```

The regular expression means:

| Expression | Meaning             |
|------------|---------------------|
| ---        | ---                 |
| ^          | Start of string     |
| \\d{2}     | Exactly two digits  |
| -          | Hyphen              |
| \\d{2}     | Exactly two digits  |
| -          | Hyphen              |
| \\d{4}     | Exactly four digits |
| \$         | End of string       |

The regex validates structure only. It does not determine whether a date exists on the calendar. For example, 31-02-2026 passes the structure check but must fail calendar validation.

## 8.4 Extract date components

```

var day = Number(transactionDate.substring(0, 2));
var month = Number(transactionDate.substring(3, 5));
var year = Number(transactionDate.substring(6, 10));

```

For 14-09-2026:

```

day    = 14
month  = 9
year   = 2026

```

## 8.5 Validate month and year

```

if (month < 1 || month > 12) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE',
    message: 'Transaction month must be between 01 and 12.'
  };
}

if (year < 1) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE',
    message: 'Transaction year must be valid.'
  };
}

```

Historical dates are accepted because the feature does not impose a minimum year.

## 8.6 Validate the day

The number of days depends on the month and year. This requires a helper function.

---

## 9. Implement getDaysInMonth()

Add this helper function to `ValidationService.gs`:

```
function getDaysInMonth(month, year) {  
  
    var daysByMonth = [  
        31, // January  
        28, // February  
        31, // March  
        30, // April  
        31, // May  
        30, // June  
        31, // July  
        31, // August  
        30, // September  
        31, // October  
        30, // November  
        31 // December  
    ];  
  
    var isLeapYear =  
        (year % 400 === 0) ||  
        (year % 4 === 0 && year % 100 !== 0);  
  
    if (month === 2 && isLeapYear) {  
        return 29;  
    }  
  
    return daysByMonth[month - 1];  
}
```

### Leap-year rule

A year is a leap year when:

- It is divisible by 400; or
- It is divisible by 4 but not divisible by 100.

Examples:

| Year | Result | Reason |
|------|--------|--------|
|------|--------|--------|

|     |     |     |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

|      |           |                |
|------|-----------|----------------|
| 2024 | Leap year | Divisible by 4 |
|------|-----------|----------------|

|      |                 |                    |
|------|-----------------|--------------------|
| 2025 | Not a leap year | Not divisible by 4 |
|------|-----------------|--------------------|

|      |           |                  |
|------|-----------|------------------|
| 2000 | Leap year | Divisible by 400 |
|------|-----------|------------------|

|      |                 |                              |
|------|-----------------|------------------------------|
| 1900 | Not a leap year | Divisible by 100 but not 400 |
|------|-----------------|------------------------------|

This ensures that:

```
29-02-2024 → Valid  
29-02-2025 → Invalid  
29-02-2000 → Valid  
29-02-1900 → Invalid
```

---

## 10. Complete ValidationService.gs Implementation

```
/**  
 * Validates a transaction date in DD-MM-YYYY format.  
 *  
 * @param {string} transactionDate  
 * @returns {Object}  
 */  
function validateTransactionDate(transactionDate) {
```

```

// 1. Date is required
if (
  transactionDate === null ||
  transactionDate === undefined ||
  transactionDate === ''
) {
  return {
    valid: false,
    code: 'TRANSACTION_DATE_REQUIRED',
    message: 'Transaction date is required.'
  };
}

// 2. Date must be a string
if (typeof transactionDate !== 'string') {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE',
    message: 'Transaction date must be a string in DD-MM-YYYY format.'
  };
}

// 3. Validate exact DD-MM-YYYY format
var datePattern = /^\\d{2}-\\d{2}-\\d{4}$/;

if (!datePattern.test(transactionDate)) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE_FORMAT',
    message: 'Transaction date must be in DD-MM-YYYY format.'
  };
}

// 4. Extract day, month, and year
var day = Number(transactionDate.substring(0, 2));
var month = Number(transactionDate.substring(3, 5));
var year = Number(transactionDate.substring(6, 10));

// 5. Validate month
if (month < 1 || month > 12) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE',
    message: 'Transaction month must be between 01 and 12.'
  };
}

// 6. Validate year
if (year < 1) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE',
    message: 'Transaction year must be valid.'
  };
}

// 7. Get the maximum number of days in the month
var daysInMonth = getDaysInMonth(month, year);

// 8. Validate day
if (day < 1 || day > daysInMonth) {
  return {
    valid: false,
    code: 'INVALID_TRANSACTION_DATE',
    message: 'Transaction date is not a valid calendar date.'
  };
}

// 9. Return the original date string
return {
  valid: true,
  value: transactionDate
};
}

/**
 * Returns the number of days in a month.
 *
 * @param {number} month
 * @param {number} year
 * @returns {number}
 */

```

```
function getDaysInMonth(month, year) {

  var daysByMonth = [
    31, // January
    28, // February
    31, // March
    30, // April
    31, // May
    30, // June
    31, // July
    31, // August
    30, // September
    31, // October
    30, // November
    31 // December
  ];

  var isLeapYear =
    (year % 400 === 0) ||
    (year % 4 === 0 && year % 100 !== 0);

  if (month === 2 && isLeapYear) {
    return 29;
  }

  return daysByMonth[month - 1];
}
```

---

## 11. Integrate Validation into TransactionService.gs

The date validator must be called before the transaction is saved.

```
const TransactionService = {

  createTransaction: function(data) {

    // 1. Validate request object
    if (!data || typeof data !== 'object') {
      return ResponseUtil.error(
        'INVALID_REQUEST',
        'Transaction request is required.'
      );
    }

    // 2. Validate transaction date
    var dateValidation = validateTransactionDate(
      data.transactionDate
    );

    // 3. Stop processing if date is invalid
    if (!dateValidation.valid) {
      return ResponseUtil.error(
        dateValidation.code,
        dateValidation.message
      );
    }

    // 4. Preserve the validated date string
    var validatedTransactionDate = dateValidation.value;

    // 5. Prepare transaction object
    var transaction = {
      id: data.id,
      transactionDate: validatedTransactionDate,
      accountId: data.accountId,
      categoryId: data.categoryId,
      type: data.type,
      amount: data.amount,
      description: data.description
    };

    // 6. Build transaction row
    var transactionRow = [
      transaction.id,
      transaction.transactionDate,
      transaction.accountId,
      transaction.categoryId,
      transaction.type,
```

```

        transaction.amount,
        transaction.description
    ];

    // 7. Save using the existing repository
    return SheetRepository.appendTransaction(transactionRow);
},

updateTransaction: function(transactionId, data) {
    // To be implemented in a future user story
},

deleteTransaction: function(transactionId) {
    // To be implemented in a future user story
},

getTransactions: function() {
    // To be implemented in a future user story
},

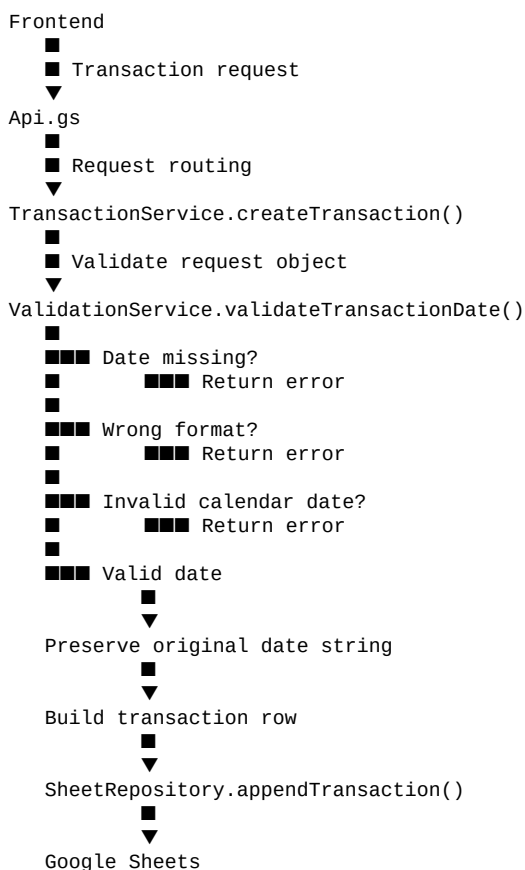
getTransaction: function(transactionId) {
    // To be implemented in a future user story
}
};

```

> **Integration note:** The transaction row shown above assumes the columns are ID, Transaction Date, Account ID, Category ID, Type, Amount, and Description. Use the actual column order defined in the project spreadsheet.

---

## 12. Complete Request Flow



The key principle is:

**Validation happens before persistence.**

---

## 13. API Request and Response Examples

### 13.1 Valid current date

Request:

```
{
  "transactionDate": "14-09-2026",
  "accountId": "ACC-001",
  "categoryId": "CAT-001",
  "type": "EXPENSE",
  "amount": 500,
  "description": "Grocery purchase"
}
```

Expected behavior:

- Date format is correct.
- Date exists on the calendar.
- Transaction proceeds to the repository.
- Date remains `14-09-2026`.

### 13.2 Valid historical date

Request:

```
{
  "transactionDate": "15-08-1995",
  "accountId": "ACC-001",
  "categoryId": "CAT-001",
  "type": "EXPENSE",
  "amount": 500,
  "description": "Historical transaction"
}
```

Expected behavior:

- Historical date is accepted.
- Date remains `15-08-1995`.
- Transaction proceeds to the repository.

### 13.3 Missing date

Request:

```
{
  "accountId": "ACC-001",
  "categoryId": "CAT-001",
  "type": "EXPENSE",
  "amount": 500
}
```

Expected response:

```
{
  "success": false,
  "error": {
    "code": "TRANSACTION_DATE_REQUIRED",
    "message": "Transaction date is required."
  }
}
```

## 13.4 Invalid format

Request:

```
{
  "transactionDate": "2026-09-14",
  "accountId": "ACC-001",
  "amount": 500
}
```

Expected response:

```
{
  "success": false,
  "error": {
    "code": "INVALID_TRANSACTION_DATE_FORMAT",
    "message": "Transaction date must be in DD-MM-YYYY format."
  }
}
```

## 13.5 Invalid calendar date

Request:

```
{
  "transactionDate": "31-02-2026",
  "accountId": "ACC-001",
  "amount": 500
}
```

Expected response:

```
{
  "success": false,
  "error": {
    "code": "INVALID_TRANSACTION_DATE",
    "message": "Transaction date is not a valid calendar date."
  }
}
```

---

# 14. Testing Strategy

Testing should cover:

1. Direct validation testing.
2. Service-level integration testing.
3. API-level testing through Postman or Bruno.
4. Storage and timezone behavior.

## 14.1 Direct validation tests

```
function testValidTransactionDate() {
  var result = validateTransactionDate('14-09-2026');
  Logger.log(result);
}

function testHistoricalTransactionDate() {
  var result = validateTransactionDate('15-08-1995');
  Logger.log(result);
}

function testInvalidTransactionDateFormat() {
  var result = validateTransactionDate('2026-09-14');
  Logger.log(result);
}
```

```
function testInvalidCalendarDate() {
  var result = validateTransactionDate('31-02-2026');
  Logger.log(result);
}

function testLeapYearDate() {
  var result = validateTransactionDate('29-02-2024');
  Logger.log(result);
}
```

## 14.2 Expected direct-validation results

Valid date:

```
{
  valid: true,
  value: '14-09-2026'
}
```

Invalid calendar date:

```
{
  valid: false,
  code: 'INVALID_TRANSACTION_DATE',
  message: 'Transaction date is not a valid calendar date.'
}
```

---

## 15. Test Case Matrix

| ID | Scenario | Input | Expected |
|----|----------|-------|----------|
|----|----------|-------|----------|

|     |     |     |     |
|-----|-----|-----|-----|
| --- | --- | --- | --- |
|-----|-----|-----|-----|

|        |                        |                    |      |
|--------|------------------------|--------------------|------|
| TC-001 | Valid current date     | `14-09-2026`       | Pass |
| TC-002 | Valid historical date  | `15-08-1995`       | Pass |
| TC-003 | Valid future date      | `31-12-2030`       | Pass |
| TC-004 | Leap-year date         | `29-02-2024`       | Pass |
| TC-005 | Non-leap-year February | `29-02-2025`       | Fail |
| TC-006 | Invalid April date     | `31-04-2026`       | Fail |
| TC-007 | Invalid February date  | `31-02-2026`       | Fail |
| TC-008 | Invalid month          | `14-13-2026`       | Fail |
| TC-009 | Zero month             | `14-00-2026`       | Fail |
| TC-010 | Zero day               | `00-09-2026`       | Fail |
| TC-011 | Single-digit day       | `4-09-2026`        | Fail |
| TC-012 | Single-digit month     | `14-9-2026`        | Fail |
| TC-013 | Wrong separator        | `14/09/2026`       | Fail |
| TC-014 | ISO format             | `2026-09-14`       | Fail |
| TC-015 | Date with time         | `14-09-2026 10:00` | Fail |
| TC-016 | Empty date             | ``                 | Fail |
| TC-017 | Null date              | `null`             | Fail |
| TC-018 | Undefined date         | `undefined`        | Fail |
| TC-019 | Numeric date           | `14092026`         | Fail |
| TC-020 | Leading space          | ` 14-09-2026`      | Fail |

TC-021 Trailing space `14-09-2026` Fail  
TC-022 Valid December date `31-12-2026` Pass  
TC-023 Invalid November date `31-11-2026` Fail  
TC-024 Century leap year `29-02-2000` Pass  
TC-025 Century non-leap year `29-02-1900` Fail

---

## 16. Definition of Done

### Validation

- Transaction date is mandatory.
- Input must be a string.
- Exact `DD-MM-YYYY` format is enforced.
- Invalid separators are rejected.
- Invalid day and month values are rejected.
- Leap-year dates are correctly handled.
- Historical dates are accepted.
- Invalid calendar dates are rejected.

### Integration

- Validation is implemented in `ValidationService.gs`.
- `TransactionService.createTransaction()` calls the validator.
- Invalid dates stop transaction processing.
- Repository is not called for invalid dates.
- Validated dates are preserved as strings.
- Existing service structure is maintained.
- Duplicate global `createTransaction()` logic is removed.

### Testing

- Direct validation tests pass.
- Historical date tests pass.
- Leap-year tests pass.
- Invalid-format tests pass.
- API-level tests pass when the transaction endpoint is available.
- No date-shifting behavior is observed.

---

## 17. File Change Summary

File Required Change

--- ---

`ValidationService.gs` Add `validateTransactionDate()`  
`ValidationService.gs` Add `getDaysInMonth()`  
`TransactionService.gs` Integrate validation into `createTransaction()`  
`TransactionService.gs` Remove duplicate global transaction-creation function  
`Config.gs` Confirm date-format configuration if applicable  
`SheetRepository.gs` No change expected  
`ResponseUtil.gs` No change expected  
`Api.gs` No change expected unless routing is incomplete  
`Code.gs` No change expected

---

## 18. Learning Summary

US-013 introduces the following concepts:

1. **Separation of concerns:** Validation belongs in `ValidationService.gs`.
2. **Regular expressions:** Used to validate the exact date structure.
3. **Calendar validation:** Used to reject impossible dates.
4. **Leap-year logic:** Used to correctly handle February.
5. **String preservation:** Prevents unnecessary timezone conversion.
6. **Service-layer validation:** Ensures the request is validated before persistence.
7. **Early return:** Stops processing when validation fails.
8. **Persistence protection:** Invalid data must never reach Google Sheets.

---

## 19. Final Outcome

After implementing US-013:

```
Valid transaction date
  ↓
Transaction can proceed

Invalid transaction date
  ↓
Transaction is rejected
```

The feature creates a reliable foundation for future transaction creation, monthly reports, date filtering, balance calculations, and historical financial analysis.